Edge cases (a task that needs no tool at all) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `no_tool_needed_practice.py`: an agent that reaches for a tool on every question wastes a full round trip each time. This checks that your loop lets the model answer directly when no tool fits. **If not:** the Build Task's "task needs no tool" Test Case would be checked for the first time there.

Run from inside `practice/`, next to `react_loop_practice.py`.

Basic Version

Approach 1 — one question, one look

```
# no_tool_needed_practice.py
from react_loop_practice import run_agent

result = run_agent("What's the capital of France?")
print("Answer:", result.final_answer)
print("Tool calls:", len(result.steps))
```

Expected output:

```
Answer: The capital of France is Paris.
Tool calls: 0
```

One run is one data point — it doesn't prove the loop behaves on other questions.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — several prompts, one `case` line each

```
# no_tool_needed_practice.py
from react_loop_practice import run_agent

# why: none of these needs add, get_weather or flaky_lookup — a model
# that calls a tool here is wasting a round trip on every question
NO_TOOL_PROMPTS = [
    "What's the capital of France?",
    "What's the boiling point of water in Celsius?",
    "Write a haiku about the sea.",
]


def check_no_tool(prompt: str) -> bool:
    result = run_agent(prompt)
    return len(result.steps) == 0


def main() -> None:
    passed = 0
    number = 0
    for prompt in NO_TOOL_PROMPTS:
        number = number + 1
        ok = check_no_tool(prompt)
        if ok:
            passed = passed + 1
        print(f"case {number} (no tool?) -> {ok}: {prompt}")
    print(f"{passed}/{len(NO_TOOL_PROMPTS)} answered without a tool")


if __name__ == "__main__":
    main()
```

Expected output:

```
case 1 (no tool?) -> True: What's the capital of France?
case 2 (no tool?) -> True: What's the boiling point of water in Celsius?
```

```
case 3 (no tool?) -> True: Write a haiku about the sea.
3/3 answered without a tool
```

Difference from Basic: three prompts instead of one, each reported, with a total — a check you can re-run after every change to a tool description.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Approach 2 — the same check as a pytest test

Story — `test_no_tool_needed.py`: pytest (`pip install pytest`) finds every function named `test_...` and runs it; `assert` stops a test with a message when its check is false. It's the same check as Approach 1, reported as pass/fail. **If not:** the Build Task's pytest version of `test_agent.py` would be the first pytest file you ever wrote.

```
# test_no_tool_needed.py
# Run it with pytest, from inside practice/: pytest test_no_tool_needed.py -v
from react_loop_practice import run_agent

NO_TOOL_PROMPTS = [
    "What's the capital of France?",
    "What's the boiling point of water in Celsius?",
    "Write a haiku about the sea.",
]


def test_general_knowledge_uses_no_tool() -> None:
    for prompt in NO_TOOL_PROMPTS:
        result = run_agent(prompt)
        # how: assert stops the test, with this message, if it's False
        assert len(result.steps) == 0, f"a tool was called for: {prompt}"


def test_answer_is_not_empty() -> None:
    result = run_agent("What's the capital of France?")
    assert result.final_answer != ""
```

Run it: `pytest test_no_tool_needed.py -v`, from inside `practice/`.

Expected output (shortened):

```
test_no_tool_needed.py::test_general_knowledge_uses_no_tool PASSED
test_no_tool_needed.py::test_answer_is_not_empty PASSED
2 passed
```

Difference from Approach 1: the same check, but pytest runs it, counts it, and prints exactly which `assert` failed and why.

Which one should you actually write? Either — Approach 1 needs nothing installed; Approach 2 is the style Doc13 uses for everything. The Build Task shows its tests both ways. If a no-tool prompt ever starts calling a tool, look at the tool's *description* first (Doc06).